

Functional Programming

Final Project

In this final project you need to develop your own game in Haskell, using the Gloss library for graphics. This is a larger and more “realistic” assignment than the previous ones, but you have a lot of freedom on what and how to implement it.

1 Introduction

In short, the goal of this final project of *Functional Programming* is to develop a small game with a bit of “action” and a bit of “intelligence” completely in Haskell. The focus is, of course, not in how great the graphics look or whether there is a charming story behind the game. Rather, the grade depends on the general architecture of the project, and in the use of good functional coding practices.

As stated above, one of the key points of this project is to architect your solution using good programming practices. Because coming up with a good design is a difficult task, we ask you to hand in a design document, stating what game you plan to implement and a rough idea of how, first. Please use the suggestions in the feedback on the design document as input to write nicer code in your final project.

We require that you work in pairs and use *git*, a version control system, to track your progress along the way. See e.g. <https://git-scm.com/docs/gittutorial> for a getting started guide with *git*. This is so that we can monitor your progress along the way and see that all team members contributed equally. Thus, remember to commit early and often.

2 Game Choice

For this project, you have to choose between one of the following four games:

1. **Asteroids:** the player controls a spaceship which can move around the screen. The spaceship has to avoid the asteroids that pop up in the screen, and can destroy them by shooting at them. After some time, different kinds of enemies try to catch them – like the asteroids, but with a bit of intelligence. In <http://www.freeasteroids.org/> you can find a very simplistic version.
2. **Shoot'em Up:** here the player also controls a spaceship, but this time it may only move vertically and shoot – this is what we call a *side-scrolling videogame*. Enemies keep coming, and the player has to avoid colliding with them or getting shot. R-Type is one game of this style, <https://www.mobygames.com/game/r-type/screenshots> offers screenshots.
3. **Pac-Man:** the player controls Pac-Man and has to eat all the circles in the maze. Meanwhile, four ghosts try to catch him. Note that each of the ghosts follow a different strategy to catch Pac-Man.
4. **Mario:** the archetypal platform game. The player controls Mario, which tries to get at the end of each level without falling in the holes or been hit by the enemies. Mario can kill those enemies by jumping over them. Usually each level ends up with a “final boss”, but you can also implement an infinite world in which enemies just keep appearing.

While these four games have roughly the same difficulty. We tried listing them in order of increasing (expected) difficulty.

3 Requirements

There are some *minimal requirements* that you game must implement in order to get a passing score for your project.

In addition, there are two ways by which you can obtain a small bonus on your final grade through the final project: half of the bonus is based on the results of your project, while the other half is based on your process.

The bonus based on the result requires having a good design and coding style, and by implementing some of the additional *optional requirements*. Note that we prefer quality over quantity here.

3.1 Minimal requirements

Player. The player controls (at least) one character, using the keyboard.

Enemies. The game has antagonistic characters in some sense, which try to make you lose the game. You should implement a minimal “intelligence” for those enemies.

Randomness. The game has to include random components: the place where new enemies appear, the place where bonus are located, and things like that.

Animation. Some events should trigger animations which span several frames. For example, destroying an enemy may create an explosion effect.

Pause. It should be possible for the player to pause the game by pressing a key of your choice.

Interaction with the file system. The program must read and write data in one or more files. You are free to decide what those files contain, a typical example would be a list of high scores.

3.2 Optional requirements

Here are some suggestions for additional features and design variations. Each of these requirements, in turn, may be developed in different degrees. Note that not all optional requirements are applicable to all four game types. This list of optional requirements is not ordered by any property, in particular not by how difficult it is to implement each feature.

Levels. The game does not have a single screen, but different ones with different characteristics. The player should be able to choose the level from a list.

Different enemies. Add multiple types of enemies, each of them with different appearance and substantially different behavior (e.g. just subtracting five rather than four hitpoints on a collision with an enemy is not sufficiently different). Make the game more interesting by making “harder” enemies appear later.

Custom levels. Develop a file format which players can use to create new levels for the game. Provide the ability to load one of those files, *in addition* to the default ones, either via a menu in the game or via the command line.

Mouse input. Allow the user to interact with the game using the mouse *in addition* to the keyboard. In contrast to keyboard actions, the effect of clicking on a place in the screen usually involves several frames until the player arrives to the selected location.

Multi-player. Either in co-operative or antagonistic mode, two players play the game at the same time. You may implement this by having each player use part of the keyboard. Be sure that the actions of one player do not kill the actions of the other one!

Networking. Implement multi-player via network communication. You may use the Network library¹. Chapter 27 of *Real World Haskell*² provides a tutorial of this package.

Complex graphics. Refine the graphical part of your game by using techniques such as *parallax scrolling*³ or by loading images from the file system. But remember: we are interested in your implementation of the techniques, not in the graphical quality *per se*.

Automated testing. Design the game in such a way that you can prove some interesting properties about it using QuickCheck.

Use JSON to save the full game state. Allow saving and loading the *full* game state to and from disk. Use JSON to serialize and deserialize the game state. You should use the Aeson⁴ library to parse and write the game state. The documentation for that library has several examples of its intended usage patterns.

3.3 Style

An important part of the bonus grade depends on the way in which you have implemented the game. Keep the following in mind while programming.

Data type definition. Think carefully about how you represent the game data in memory. Create different data types for different goals, as explained during the lectures.

Use abstraction. Remove duplication in your code by means of (higher-order) functions. Use type classes to implement functionality common to several data types.

Separate pure and impure parts. Do not put all functionality in *IO*. Most of the rendering and update functions can be written in a pure style.

Modularity. Split your code into functions, and those functions into modules. Keep their size under control. Feel free to introduce as many modules as required.

Follow good practices. Try to use higher-order functions such as *foldr* or *map* instead of recursion, use pattern matching to define equations, do not use magic numbers.

Document your code. Always write type signatures for all your functions. Explain complex or subtle parts of your program in comments.

3.4 The Process

You can obtain some bonus points by having two progress meetings with your TA, during which you show that you have made appropriate progress and that you and your team-mate have collaborated, as evidenced by somewhat balanced contributions in the git repository.

¹<http://hackage.haskell.org/package/network>

²<http://book.realworldhaskell.org/read/sockets-and-syslog.html>

³https://en.wikipedia.org/wiki/Parallax_scrolling

⁴<http://hackage.haskell.org/package/aeson/docs/Data-Aeson.html>

The first short meeting with the TA should take place by October 20. We expect at least one genuine question that shows that you have put thought into the assignment and your design, as well as significant progress towards some first minimal requirements.

The second meeting should take place by November 6. We similarly expect at least one genuine question, and you should be able to explain a part of your code selected by the TA.

These meetings are expected to take only a few minutes. Arrange them directly with a TA, possibly during one of the practical sessions.

4 Libraries and tooling

We provide an example project using Gloss which just updates the screen with a random number every now and then. You can use this example project as a template for your own code or initialize a new one following the steps explained during the lectures. Make sure to create and initialize an initial git repository. In any case, please ensure that you can run your project with the files you submit to us.

4.1 The Gloss library

The Gloss library provides a nice interface to build small 2-D games. It is also a good example of a domain-specific language; in this case for the definition of what to show on the screen. In order to define a game in Gloss you need to write four functions, all of them operating on a shared *state of the world* w , which you ought to define.

- What is the initial state of the world.
- How to turn the world w into a *Picture*, which is then displayed in the screen.
- How to handle user input in the form of *Events*.
- How to update the state of the world every time some amount of seconds have elapsed.

This design is commonly known as *model-view-controller*. The model consists of the definitions of the involved data types; the view is the function which displays the picture; and the controller how to update that state in response to either time or user interaction. In the example project, each of these components is defined in a different module.

Gloss provides a pure interface. Alas, this makes it difficult to have random elements in the game, and impossible to access the file system. For that reason we suggest you to use the *IO* version of Gloss, which can be found in the module *Graphics.Gloss.Interface.IO.Game*. The key function is *playIO*, which after a bunch of configuration options receives the four aforementioned functions,

```
initial :: world
view    :: world → IO Picture
input   :: Event → world → IO world
step    :: Float → world → IO world
```

You can look at the *main* function for an example of how to initialize Gloss.

The *Picture* data type, defined in the *Graphics.Gloss.Data.Picture* module,⁵ provides several functions to build the entire scene of your game:

- Primitive shapes, such as *circle*, *polygon*, and *text*.
- To change the color of a picture, use the function *color* with one of the colors defined in the module *Graphics.Gloss.Data.Color*. For example, here is the code to display a green A:

```
color green (text "A")
```

- Transformations over a picture, like *translate* and *rotate*.

⁵<https://hackage.haskell.org/package/gloss/docs/Graphics-Gloss-Data-Picture.html>

- Finally, a way to compose several pictures into another one. Note that pictures are drawn on top of each other, so you need to translate them first. For example:

```
picture [text "A", translate 20 20 (color green (circle 10))]
```

A neat way to explore drawing with Gloss is by means of an interactive session with the interpreter.

```
Prelude> import Graphics.Gloss
Prelude Graphics.Gloss> let d = InWindow "example" (800, 600) (0, 0)
Prelude Graphics.Gloss> display d black (color green (circle 100))
```

4.2 Using records

In the lectures you have learnt to access information inside a value by using *pattern matching* and to build new values by calling the *constructors* of the data type. This is very simple, but breaks when we need to handle larger data types with several levels of nesting, as will be the case with the state of your game. For this scenario, Haskell provides a nice feature called *records*.

Definition. A record is like any other Haskell data type, with an additional piece of information: a name for each of the fields. For example, here is the definition of *GameState* in the example project:

```
data GameState = GameState {
    infoToShow :: InfoToShow
    , elapsedTime :: Float
}
```

For comparison, here is the definition without field names:

```
data GameState = GameState InfoToShow Float
```

Accessing information in a record. Pattern matching is available as usual for records,

```
view (GameState info elapsed) = ...
```

In addition, records introduce other *three* forms of accessing the information:

1. We can also use the name of the field as a function which retrieves that piece of information,

```
infoToShow :: GameState → InfoToShow
elapsedTime :: GameState → Float
```

For example, in the example project we are only interested in the first field to display:

```
view gstate = case infoToShow gstate of ...
```

2. Pattern matching is extended to retrieve the information of a field using a key/value syntax:

```
view GameState { infoToShow = info } = ... -- use 'info' here
```

The great advantage is that we only need to include those fields we are interested in, whereas with usual pattern matching we need to give names (or use underscores) for *every* field.

3. We can even make it shorter if we enable the “named field puns” extension by including the following line at the beginning of your file:

```
{-# language NamedFieldPuns #-}
```

Then you do not even need to use *field = value* in pattern matching, only the name of the field:

```
view GameState { infoToShow } = ... -- use 'infoToShow' here
```

Updating part of a record. Another problem with normal data types is that if you want to change only one part of a value you are forced to write all the other fields in your source. With records you can tell the compiler to reuse part of a previous record and only “update” one or more fields.

```
step secs gstate = gstate { elapsedTime = elapsedTime gstate + secs }
```

In this example, we want to keep the whole game state – we do not even match on it – except for the *elapsedTime*, which is increased by the given amount of seconds. An alternative way to write this function is:

```
step secs gstate@(GameState { elapsedTime }) = gstate { elapsedTime = elapsedTime + secs }
```

In this case the final *elapsedTime* refers to the value in the original game state, which we obtain using the named field syntax, whereas the other appearances refer to the field name.

Lenses. The problem of accessing larger and nested pieces of data has been “itchy” issue for Haskellers since a long time ago. Records are a simple solution, but many other people use a more powerful notion called *lenses*. If you want to dive into it – but not before having a working game! – look at the packages `lens-tutorial` and `microlens`.

4.3 Loading bitmaps

It is strongly recommended to start by implementing your game with simple pictures, like squares and circles. But if you want to make it look better you can load bitmaps and use them in Gloss. (However, this will not influence your result score).

The only function you need is `loadBMP :: String → IO Picture`. This function receives the path to a file, relative to your current folder, and loads it into memory. Since the result is a *Picture*, you can use it in the same way as you would use any other shape. Note however that it works on *IO*, so you should plan ahead the initialization of the pictures, before moving into the pure world.

Another important detail is that `loadBMP` only supports images in BMP format. If your files are in other format, like JPEG, PNG, or GIF, you need to convert them first to BMP. There are plenty of tools which can perform this work; as the assistants if you need help.

Finally, remember that you should **not** use material which falls under copyright, especially if you are hosting your code somewhere else, like the UU servers or GitHub. `OpenGameArt`, <https://opengameart.org/>, provides a huge amount of graphical resources that you can use for free.

5 Getting Started

There are two parts to this final project, that you will have to hand in at different moments:

- An initial design document, and
- The implementation of your final game (with updated design document if necessary).

You have to submit your work via Brightspace. See the course webpage for the exact deadlines.

5.1 The Design Document

You *have to* submit a design document briefly describing your game. We are mostly interested in how you plan to structure your game, rather than in how pretty your game will look, or how fun it will be to play it. In particular, make sure your design document answers the following questions.

1. Which game type will you implement?
2. If you plan on having a “twist” with respect to the basic games, briefly (at most one paragraph) describe it.
3. What data types will you use?

4. How will (the architecture of) your game incorporate the minimal requirements?
For each of the minimal requirements, specify briefly how your game will incorporate it. In particular, how will you structure your code to support this requirement. For example, for randomness: how/where do you plan to incorporate it, how is this reflected in your data types?
5. Which optional requirements are you planning to implement? For each of them, specify how you plan to structure your code and data types to support them, as you did with the minimal requirements.
6. How will you separate the pure and impure parts in your game?
7. Do you foresee opportunities for using abstraction? For example, is there functionality that you will want to implement using typeclasses?

Try to be specific; a good design document will already describe the data types you plan to use in a fair amount of detail. To describe you will implement the requirements: consider giving the function names with their type signatures. An example design document is available on the website.

Remember to **avoid using an “object oriented” approach** that does something like:

```
data Object = Mario Position | Mushroom Position | Ground | Flower | ...
type World = [Object]
```

as it does *not* provide sufficient type safety. For example, it does not differentiate between Mario and the environment, nor does it enforce that there is only one Mario (rather than many). Hence, starting with one generic “object” type and refining it in “subclasses” is bad practice in a functional language!

When you provide snippets and/or function signatures make sure they are highlighted properly. This makes it easier to read for everyone.

What to hand in: A single pdf file describing your game.

5.2 The Implementation

Use the starting framework available on the website to implement your game.

What to hand in: A single zip file which contains:

- The original design document,
- an updated design document that reflects the game as it was implemented,
- a README with instructions on how to run and play the game,
- The source code (and all required assets) as a *buildable* Cabal project. Make sure that you can properly build/compile your game just using ‘cabal build’ or ‘stack build’ from within the folder containing your cabal file.

Please do *not* include the `dist-newstyle` folder.

- The `.git` folder (this will make sure we can see how you made progress on the game during the project).

Make sure to upload *and submit* your game correctly on Brightspace.